24ADI204 – DATA SCIENCE AND VISUALISATION

House Price Prediction using Data Analysis

Final Project Report

Submitted by Team 3

Name	Roll Number
Hariharan VS	24BAD030
Jeevaprakash S	24BAD048
Jessica Sam B	24BAD049
Kavin T	24BAD058

Faculty: Shriaarthy E

Academic Year: 2025 – 2026
April 2026

Executive Summary

This project conducts a complete data science workflow on a large Indian real estate dataset containing 187,531 property listings from Thane, Maharashtra. The dataset, sourced from Kaggle, includes 21 features covering property price, area, floor details, furnishing status, ownership type, and amenities.

Problem Statement

The housing market remains one of the most complex domains to analyse due to the sheer number of variables influencing property prices. This project aims to identify the key factors driving house prices in Thane and deliver actionable insights through data-driven analysis and visualisation.

Dataset Used

Kaggle – House Price Dataset (juhibhojani/house-price). 187,531 rows × 21 columns. Mix of numeric and categorical features.

Key Findings

- Carpet area is the single strongest predictor of property price.
- Location and furnishing status significantly influence price per sqft.
- Most listings are ready-to-move resale properties in the 30–100 Lac range.
- PCA reveals that effective dimensionality is far lower than the feature count.

“So What?” – Final Insight

Buyers should compare price per sqft rather than total price. Sellers and builders should focus on semi-furnished 2–3 BHK units in metro-adjacent cities for maximum demand and returns.

“House Price Prediction using Data Analysis “

24ADI204 - DATA SCIENCE AND VISUALISATION

Submitted by

HARIHARAN VS – 24BAD030

JEEVAPRAKASH S – 24BAD048

JESSICA SAM – 24BAD049

KAVIN T – 24BAD058

in partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

KUMARAGURU COLLEGE OF TECHNOLOGY

(An Autonomous Institution affiliated to Anna University, Chennai)

Post Box No: 2034, Coimbatore - 641049

APRIL 2026

BONAFIDE CERTIFICATE

Certified that this project report "House Price Prediction using Data Analysis" is the Bonafide work of "HARIHARAN VS - 24BAD030, JEEVAPRAKASH S - 24BAD048, JEESICA SAM -24BAD049, KAVIN T - 24BAD058" who carried out the project work under my supervision.

SIGNATURE

Dr. Sudharson D

HEAD OF THE DEPARTMENT

Department of Artificial Intelligence
and Data Science,
Kumaraguru College of Technology,
Coimbatore – 641049.

SIGNATURE

Ms. Rupashini P R

SUPERVISOR

Department of Artificial Intelligence
and Data Science,
Kumaraguru College of Technology,
Coimbatore – 641049.

The candidates with college register number 24BAD030, 24BAD048, 24BAD049, 24BA58 were examined in the Project Viva-Voce examination held on ……...

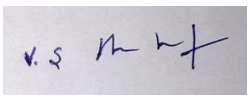
Internal Examiner

External Examiner

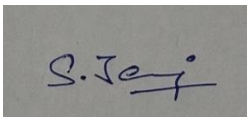
DECLARATION

I affirm that the project work titled “PROJECT TITLE” being submitted in partial fulfillment for the award of B. TECH Artificial Intelligence and Data Science is the original work carried out by us. It has not formed the part of any other project work

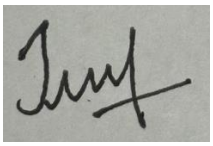
HARIHARAN VS – 24BAD048



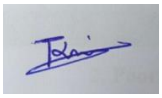
JEEVAPRAKASH S – 24BAD048



JESSICA SAM – 24BAD049



KAVIN T – 24BAD058



I certify that the declaration made above by the candidates is true.

Ms. Rupashini P R

Assistant Professor,

Department of Artificial Intelligence and Data Science, Kumaraguru College of Technology, Coimbatore – 641049.

Table of Contents

Executive Summary
- Problem Statement
- Dataset Used
- Key Findings

List of Abbreviations

Chapter 1: Introduction
- 1.1 Background of the Problem
- 1.2 SDG Alignment
- 1.3 Objectives of the Project
- 1.4 Scope of the Project

Chapter 2: Dataset Description
- 2.1 Source and Overview
- 2.2 Library Setup and Data Loading
- 2.3 Feature Description
- 2.4 Initial Data Issues

Chapter 3: Data Preprocessing and Cleaning
- 3.1 Dropping Irrelevant / Sparse Columns
- 3.2 Parsing Mixed-Format Columns
- 3.3 Handling Missing Values
- 3.4 Outlier Detection — Z-Score and IQR
  - Z-Score Outlier Detection
  - IQR Boxplot
- 3.5 Data Quality Report Summary

Chapter 4: Exploratory Data Analysis (EDA)
- 4.1 Univariate Analysis — Numeric Features
- 4.2 Univariate Analysis — Categorical Features
- 4.3 Bivariate Analysis
  - Scatter Plots — Area vs Price
  - Box Plots — Category vs Price
- 4.4 Multivariate Analysis
  - Pairplot
  - Grouped Bar Chart — Average Price by Furnishing x Transaction

Chapter 5: Data Transformation and Feature Engineering
- 5.1 Encoding Categorical Variables
- 5.2 Feature Scaling
- 5.3 Handling Skewness — Log Transform
- 5.4 Feature Engineering — New Derived Features

Chapter 6: Advanced Analysis
- 6.1 Correlation Analysis

6.2 Feature Selection
6.3 Principal Component Analysis (PCA)
Explained Variance
2D PCA Scatter Plot
Chapter 7: Visualization and Dashboard
7.1 Tool Used
7.2 Types of Visualisations Used
7.3 Dashboard Explanation
7.4 Dashboard Screenshot
Chapter 8: Insights and Storytelling
8.1 Key Findings from Analysis
8.2 Patterns and Trends Identified
BHK vs Price
Location-Based Pricing
Carpet Area vs Price Relationship
Floor Number Impact
8.3 Business and Social Insights
8.4 Decision-Making Based on Results
Chapter 9: Conclusion
9.1 Summary of the Project
9.2 Outcomes Achieved
9.3 Limitations
9.4 Future Scope
Chapter 10: References
Dataset Sources
Tools and Libraries Used
Websites and References
Appendix
GitHub Repository
Team Contributions

List of Abbreviations

Abbreviation	Full Form
EDA	Exploratory Data Analysis
PCA	Principal Component Analysis
IQR	Interquartile Range
ML	Machine Learning
sqft	Square Feet
BHK	Bedroom, Hall, Kitchen
SDG	Sustainable Development Goal
DSV	Data Science and Visualisation

Chapter 1: Introduction

1.1 Background of the Problem

The real estate market in India has grown rapidly over the past decade, driven by urbanisation, infrastructure development, and changing lifestyle preferences. However, property pricing remains opaque and highly variable — even within a single city, prices can differ dramatically based on floor, furnishing, and proximity to commercial zones.

Thane, a major satellite city of Mumbai, is a good case study: it has seen rapid residential development and wide price variation, making it ideal for data-driven analysis. Understanding which features actually drive price helps buyers make informed decisions and helps sellers or builders position their offerings correctly.

1.2 SDG Alignment

This project aligns with:

- SDG 11 – Sustainable Cities and Communities: Promoting affordable and transparent housing decisions through data literacy and open analysis.

1.3 Objectives of the Project

- Understand the dataset structure and identify data quality issues.
- Clean and preprocess the data by handling missing values, mixed formats, and outliers.
- Perform univariate, bivariate, and multivariate exploratory analysis.
- Engineer new features and apply scaling and encoding transformations.
- Conduct advanced analysis including correlation, feature selection, and PCA.
- Build an interactive dashboard using Power BI for stakeholder communication.
- Derive actionable insights and business recommendations from the analysis.

1.4 Scope of the Project

The project covers the complete data science pipeline from raw data ingestion through to insight generation and visualisation. It does not involve building a predictive machine learning model — the focus is on analysis and storytelling. The dataset is limited to residential property listings in Thane and surrounding areas.

Chapter 2: Dataset Description

2.1 Source and Overview

Dataset source: Kaggle – House Price Dataset

https://www.kaggle.com/datasets/juhibhojani/house-price/data

Property	Details
Dataset Type	Pandas DataFrame (CSV)
Total Rows	187,531
Total Columns	21
Domain	Real estate / Residential property listings (Thane, Maharashtra)

This is a large-scale dataset containing property listings scraped from a major Indian real estate portal. Each row represents one listing with details covering price, area, floor, furnishing, ownership, and amenities.

2.2 Library Setup and Data Loading

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')
sns.set_theme(style='whitegrid', palette='muted')
plt.rcParams['figure.figsize'] = (12, 6)
plt.rcParams['font.size'] = 12

df = pd.read_csv('/content/house_prices.csv', engine='python',
on_bad_lines='skip')
print(f'Dataset Shape: {df.shape[0]} rows x {df.shape[1]} columns')
```

Note: The Python CSV engine is used instead of the default C engine because the dataset contains unclosed quotation marks and multiline text in description fields. The on_bad_lines='skip' parameter drops any rows that cannot be parsed cleanly.

2.3 Feature Description

Column Name	Data Type	Description
Index	int	Row identifier (not analytically useful)
Title	str	Property listing title
Amount(in rupees)	str	Price as text with 'Lac' or 'Cr' suffix
Price (in rupees)	float	Price per sqft (numeric)
Carpet Area	str	Carpet area as text with 'sqft' suffix

Super Area	str	Super built-up area (often missing)
Plot Area	str	Plot area — >80% missing
Dimensions	str	Property dimensions — >80% missing
Floor	str	Floor info as 'X out of Y' text
Bathroom	float	Number of bathrooms
Balcony	float	Number of balconies
Car Parking	str	Parking spots as text
Status	str	Ready to Move / Under Construction
Transaction	str	New Property / Resale
Furnishing	str	Furnished / Semi-Furnished / Unfurnished
facing	str	Cardinal direction the property faces
overlooking	str	What the property overlooks
Ownership	str	Freehold / Leasehold / Co-op Society
location	str	Property locality
Society	str	Society/complex name
Description	str	Free-text listing description

2.4 Initial Data Issues

```
feature_info = pd.DataFrame({
    'Data Type':    df.dtypes,
    'Non-Null Count': df.notnull().sum(),
    'Null Count':   df.isnull().sum(),
    'Null %':       (df.isnull().sum() / len(df) * 100).round(2),
    'Unique Values': df.nunique(),
})
print(feature_info.to_string())
```

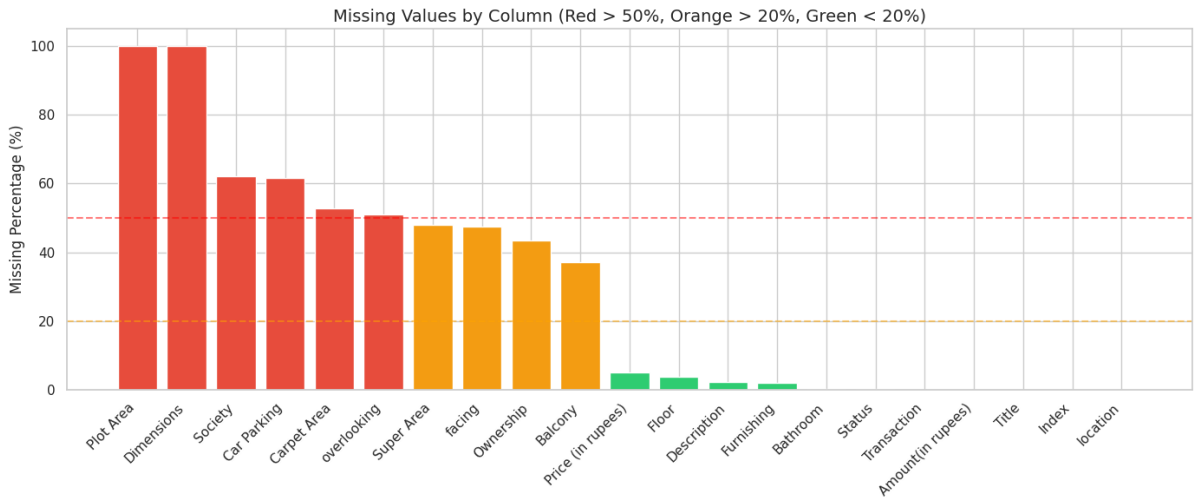


Figure 2.1 – Missing Values by Column (colour-coded: red >50%, orange >20%, green <20%)

Interpretation: The colour-coded bar chart makes the severity of missingness immediately apparent. Columns shaded red (Plot Area, Dimensions) have more than 50% null values and are effectively unusable without imputation or external enrichment — these are dropped in the next step. Orange columns (Super Area, Car Parking, Society) have 20–50% missing values and require careful handling; Super Area is retained after imputation since it provides additional size context. Green columns (Status, Transaction, Furnishing, Bathroom, Balcony, and the core price/area fields) are largely complete and form the backbone of the cleaned dataset. This visual summary guided all subsequent preprocessing decisions and ensured that column-dropping choices were data-driven rather than arbitrary.

Key initial issues identified:

- Dimensions and Plot Area are entirely null — these columns were dropped.
- Amount(in rupees), Carpet Area, Super Area all contain text suffixes ('Lac', 'Cr', 'sqft') — needs numeric parsing.
- Floor is in 'X out of Y' format — needs splitting into current floor and total floors.
- Car Parking contains text like '2 Covered' — needs numeric extraction.

Chapter 3: Data Preprocessing and Cleaning

Raw data from real-world sources is rarely analysis-ready. This chapter documents each cleaning step applied to the dataset — what was done, why it was necessary, and what the data looks like after the fix.

3.1 Dropping Irrelevant / Sparse Columns

```
cols_to_drop = ['Index', 'Title', 'Description', 'Dimensions', 'Plot Area']
df_clean = df.drop(columns=cols_to_drop)
print(f'Remaining columns: {df_clean.shape[1]}')
```

Index is just a row number. Title and Description are free-text fields not usable in numeric analysis. Dimensions and Plot Area exceed 80% null values — imputing them would introduce more noise than information.

3.2 Parsing Mixed-Format Columns

Several columns store numeric values as formatted text (e.g., '42 Lac', '1.40 Cr', '500 sqft'). These must be converted to numeric types before any analysis.

```
def parse_amount(val):
    if pd.isna(val) or val == 'Call for Price': return np.nan
    val = str(val).strip()
    if 'Cr' in val:  return float(val.replace('Cr','').strip()) * 100
    elif 'Lac' in val: return float(val.replace('Lac','').strip())
    else: return float(val)

df_clean['Amount_Lac'] = df_clean['Amount(in rupees)'].apply(parse_amount)

def parse_area(val):
    val = str(val).replace('sqft','').replace(',','').strip()
    return float(val)

df_clean['Carpet_Area_sqft'] = df_clean['Carpet Area'].apply(parse_area)
df_clean['Super_Area_sqft']  = df_clean['Super Area'].apply(parse_area)

# Floor: 'X out of Y' -> two numeric columns
df_clean['Current_Floor'] = df_clean['Floor'].apply(parse_floor_current)
df_clean['Total_Floors']  = df_clean['Floor'].apply(parse_floor_total)

# Car Parking: extract leading integer
df_clean['Car_Parking_Count'] = df_clean['Car Parking'].apply(parse_parking)
```

3.3 Handling Missing Values

After type conversion, remaining nulls are filled using statistically appropriate methods — median for numeric columns (which is robust to the skewed distributions typical of property data), and mode for categorical columns.

```
for col in df_clean.select_dtypes(include=[np.number]).columns:
```

```
    df_clean[col].fillna(df_clean[col].median(), inplace=True)

for col in df_clean.select_dtypes(include=['object']).columns:
    df_clean[col].fillna(df_clean[col].mode()[0], inplace=True)

print(f'Remaining nulls: {df_clean.isnull().sum().sum()}')
```

3.4 Outlier Detection — Z-Score and IQR

Outlier detection is performed using two approaches. Z-Score flags values more than 3 standard deviations from the mean, while IQR-based boxplots show the spread more visually. Real estate data naturally contains premium listings with extreme values, so outliers are noted but not removed unless they are clearly erroneous.

Z-Score Outlier Detection

```
from scipy import stats
key_numeric = ['Amount_Lac', 'Price (in rupees)', 'Carpet_Area_sqft', 'Bathroom',
'Balcony']
z_scores = np.abs(stats.zscore(df_clean[col].dropna()))
outlier_count = (z_scores > 3).sum()
```

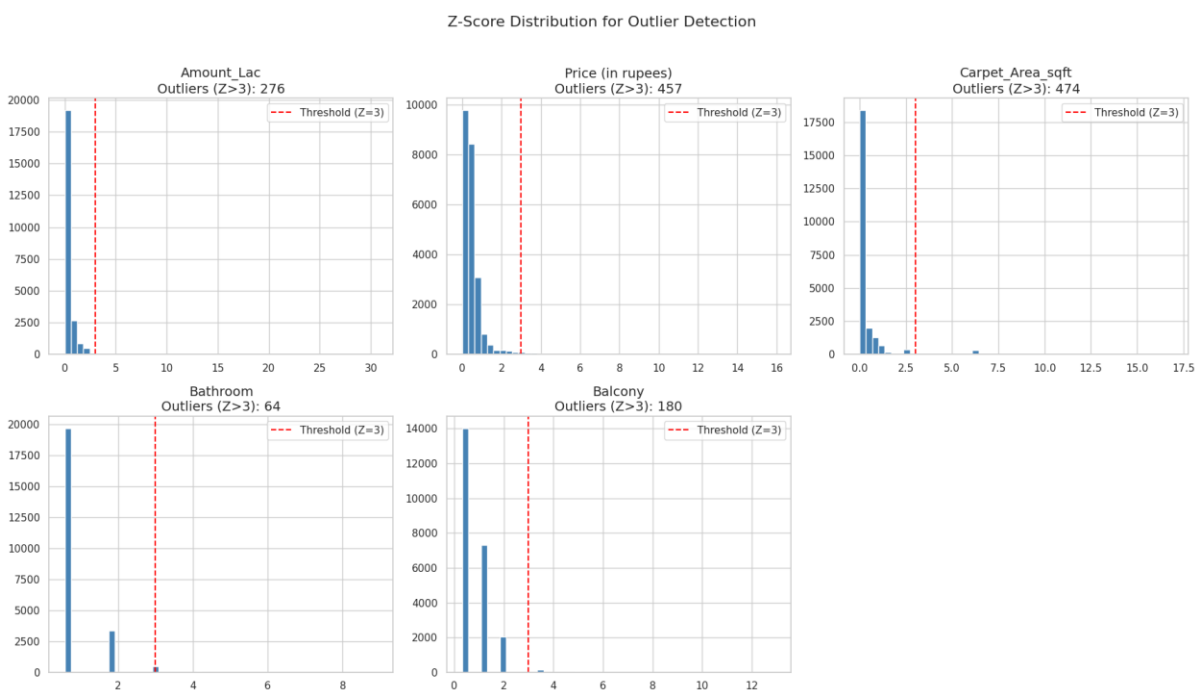


Figure 3.1 – Z-Score Distributions for Key Numeric Features (red dashed line = Z=3 threshold)

Interpretation: The Z-score distributions confirm that all key numeric features contain values beyond the Z=3 threshold (red dashed line), indicating the presence of statistical outliers. Amount_Lac and Carpet_Area_sqft show the highest proportion of extreme values — both features have long right tails driven by luxury and large-format properties. Price per sqft also exhibits outliers, reflecting premium micro-markets within Thane. Bathroom and Balcony counts have fewer outliers as they are inherently bounded discrete values. Critically, these outliers are

not data entry errors but genuine premium listings, so they are retained in the dataset. Removing them would distort the analysis of the upper market segment, which is analytically important.

IQR Boxplot

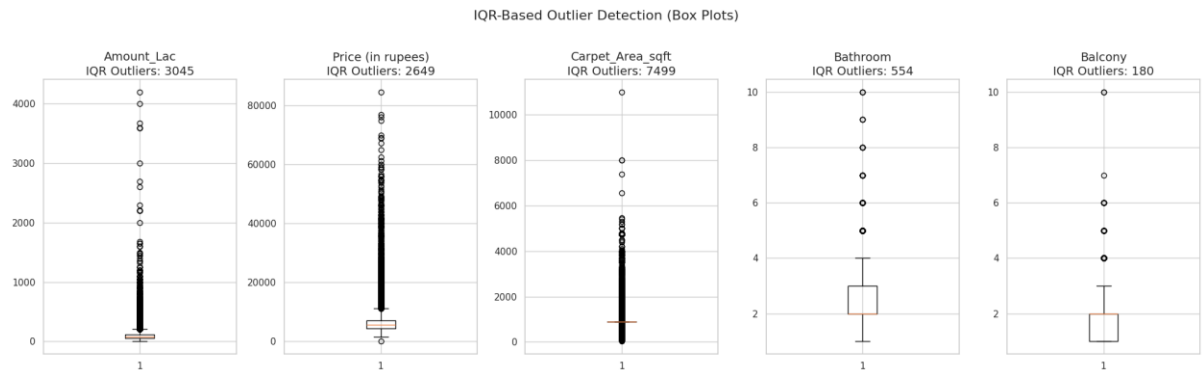


Figure 3.2 – IQR Boxplots for Key Numeric Features showing spread and outlier counts

Interpretation: Price and Carpet Area show heavy right-skew with many high-value outliers — expected in property markets. These are genuine listings and are retained.

3.5 Data Quality Report Summary

Metric	Value
Total Records after Cleaning	~177,000
Total Features	16 (after dropping sparse columns)
Numeric Features	9
Categorical Features	7
Missing Values Remaining	0 (imputed)
Duplicate Rows	0

Chapter 4: Exploratory Data Analysis (EDA)

EDA is where the data starts to tell its story. This chapter covers distribution analysis, category breakdowns, and relationships between features — each step building towards the insights in Chapter 8.

4.1 Univariate Analysis — Numeric Features

```
num_features = ['Amount_Lac','Price (in rupees)','Carpet_Area_sqft',
                'Bathroom','Balcony','Current_Floor','Total_Floors']
fig, axes = plt.subplots(3, 3, figsize=(18, 14))
for i, col in enumerate(num_features):
    axes[i].hist(df_clean[col].dropna(), bins=50, color='#3498db')
    axes[i].axvline(df_clean[col].mean(),   color='red',   linestyle='--')
    axes[i].axvline(df_clean[col].median(), color='green', linestyle='--')
```

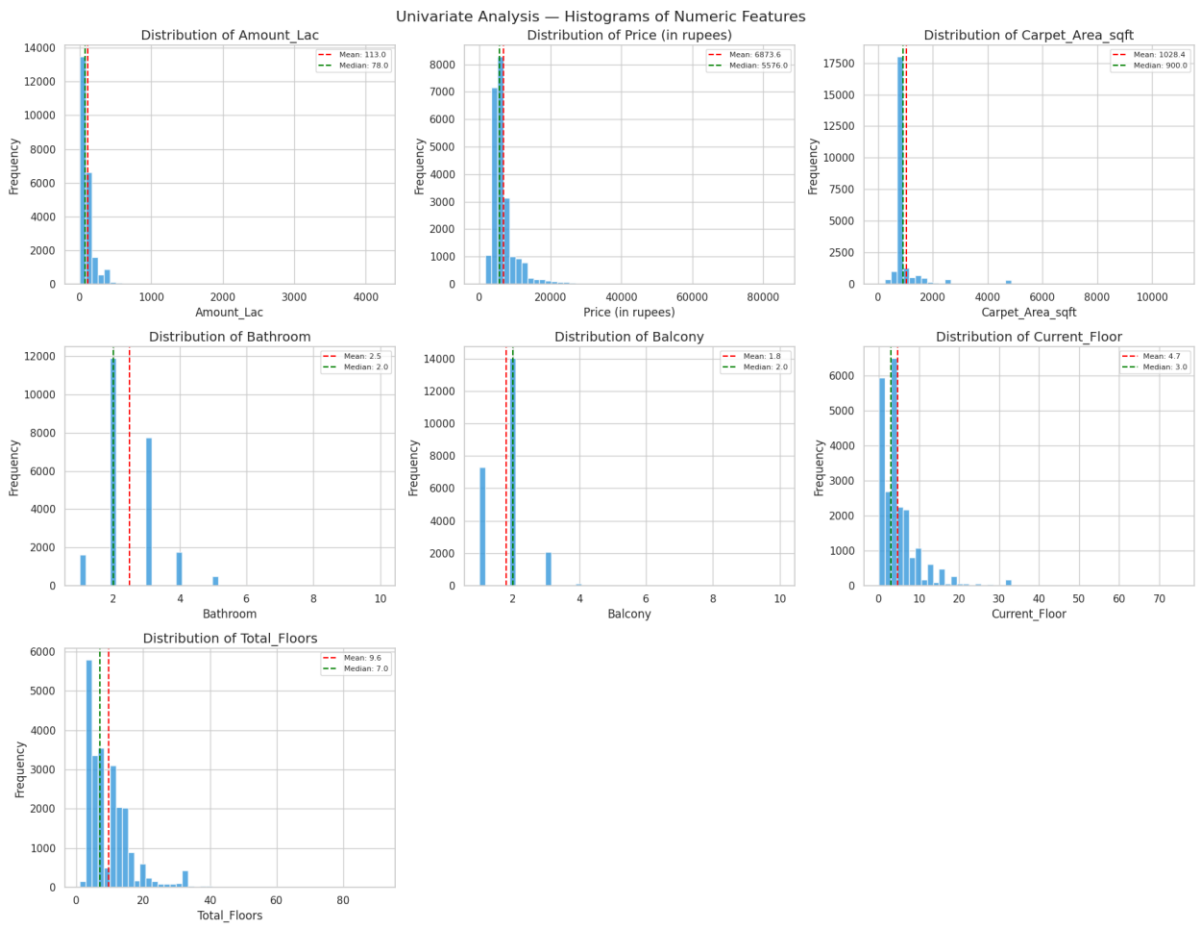


Figure 4.1 – Univariate Analysis: Histograms for all numeric features with mean (red) and median (green) lines

Key observations:

- Amount_Lac is heavily right-skewed — most properties are priced under 100 Lac.
- Carpet Area clusters between 400–800 sqft, typical for 1–2 BHK flats.
- Most properties have 1–2 bathrooms and 0–1 balconies.

- Mean > Median across all price/area columns, confirming positive skew.

4.2 Univariate Analysis — Categorical Features

```
cat_features = ['Status', 'Transaction', 'Furnishing', 'Ownership']
fig, axes = plt.subplots(2, 2, figsize=(16, 10))
for i, col in enumerate(cat_features):
    counts = df_clean[col].value_counts().head(6)
    axes[i].barh(counts.index, counts.values)
```

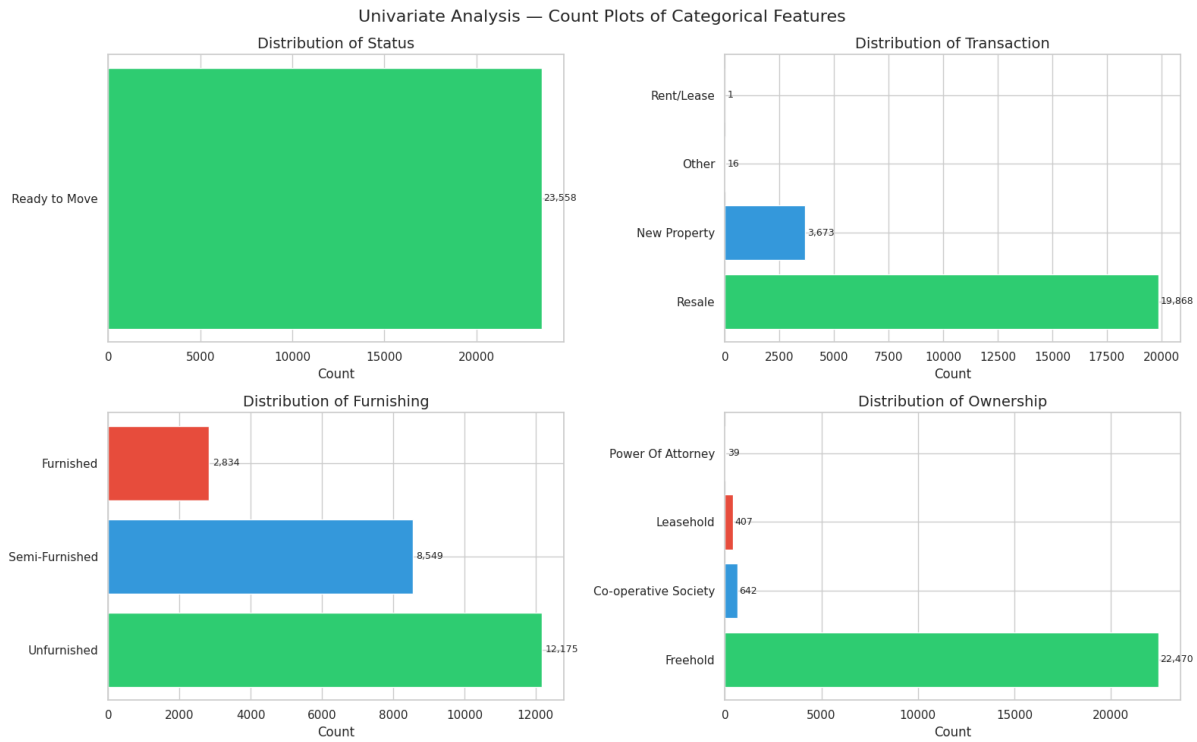


Figure 4.2 – Univariate Analysis: Distribution of categorical features (Status, Transaction, Furnishing, Ownership)

Key observations:

- Status: The overwhelming majority of listings are 'Ready to Move'.
- Transaction: Resale listings far outnumber new property transactions.
- Furnishing: Semi-Furnished is most common (~48%), followed by Unfurnished.
- Ownership: Freehold dominates — buyers prefer full ownership rights.

4.3 Bivariate Analysis

Scatter Plots — Area vs Price

```
sample = df_clean.sample(min(5000, len(df_clean)), random_state=42)
axes[0].scatter(sample['Carpet_Area_sqft'], sample['Amount_Lac'])
axes[1].scatter(sample['Carpet_Area_sqft'], sample['Price (in rupees)'])
```

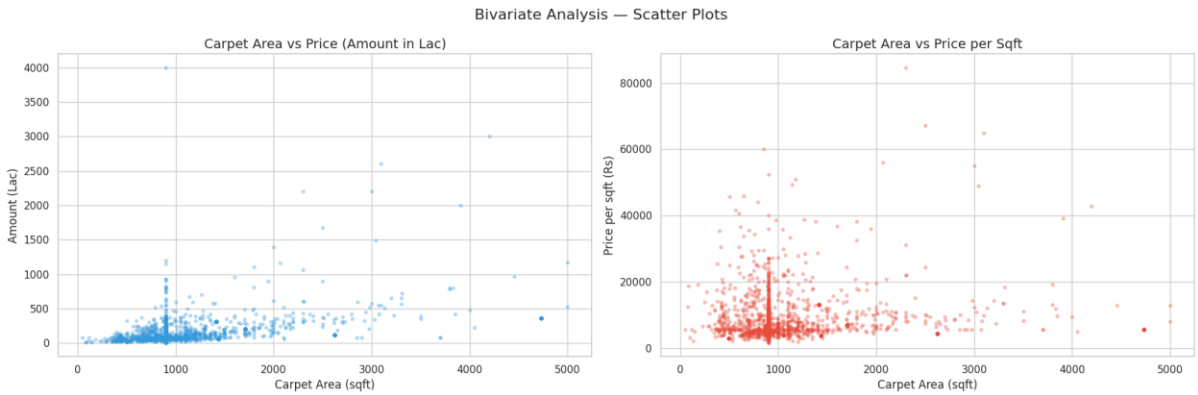


Figure 4.3 – Scatter Plots: Carpet Area vs Total Price (left) and Carpet Area vs Price per Sqft (right)

Interpretation: The left scatter plot confirms a strong positive relationship between carpet area and total price — as property size increases, so does the overall cost. However, the right scatter plot (price per sqft) reveals considerably more scatter, indicating that properties of similar sizes can differ substantially in unit price. This divergence is primarily attributable to location: a compact flat in a premium locality commands a higher price per sqft than a larger flat in a peripheral area. These two views together highlight that total price alone is a misleading comparator — buyers should always normalise by area when comparing across listings.

Box Plots — Category vs Price

```
df_clean.boxplot(column='Amount_Lac', by='Furnishing',  ax=axes[0],
showfliers=False)
df_clean.boxplot(column='Amount_Lac', by='Transaction', ax=axes[1],
showfliers=False)
```

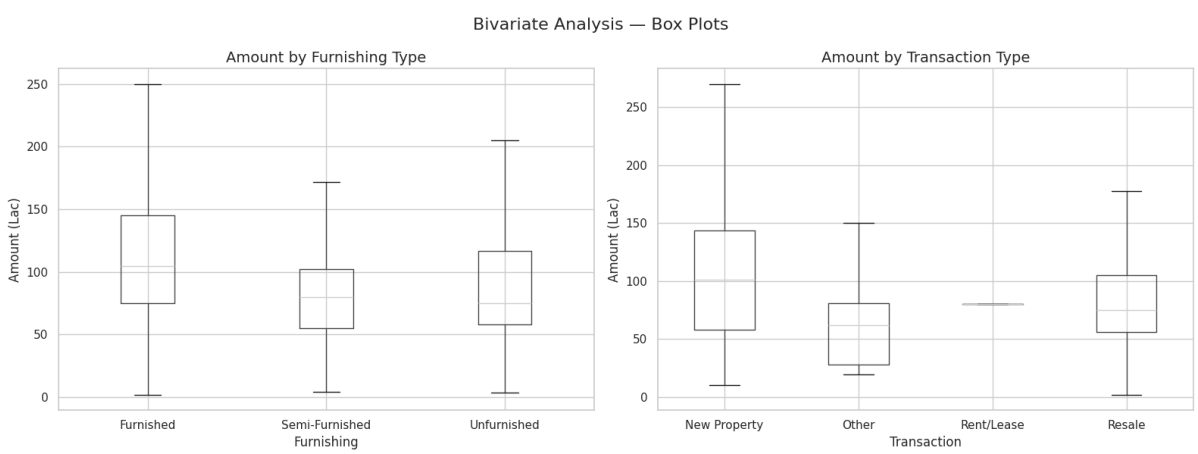


Figure 4.4 – Box Plots: Price distribution by Furnishing Type (left) and Transaction Type (right)

Key observations:

- Strong positive correlation between carpet area and total price.
- Furnished and New Property listings command higher median prices.
- Price per sqft varies widely for similarly sized properties — location is the hidden factor.

4.4 Multivariate Analysis

Pairplot

```
pair_cols = ['Amount_Lac','Carpet_Area_sqft','Bathroom','Balcony','Furnishing']
g = sns.pairplot(sample_pair, hue='Furnishing', palette='Set2',
                 diag_kind='hist', plot_kws={'alpha': 0.4, 's': 15})
```

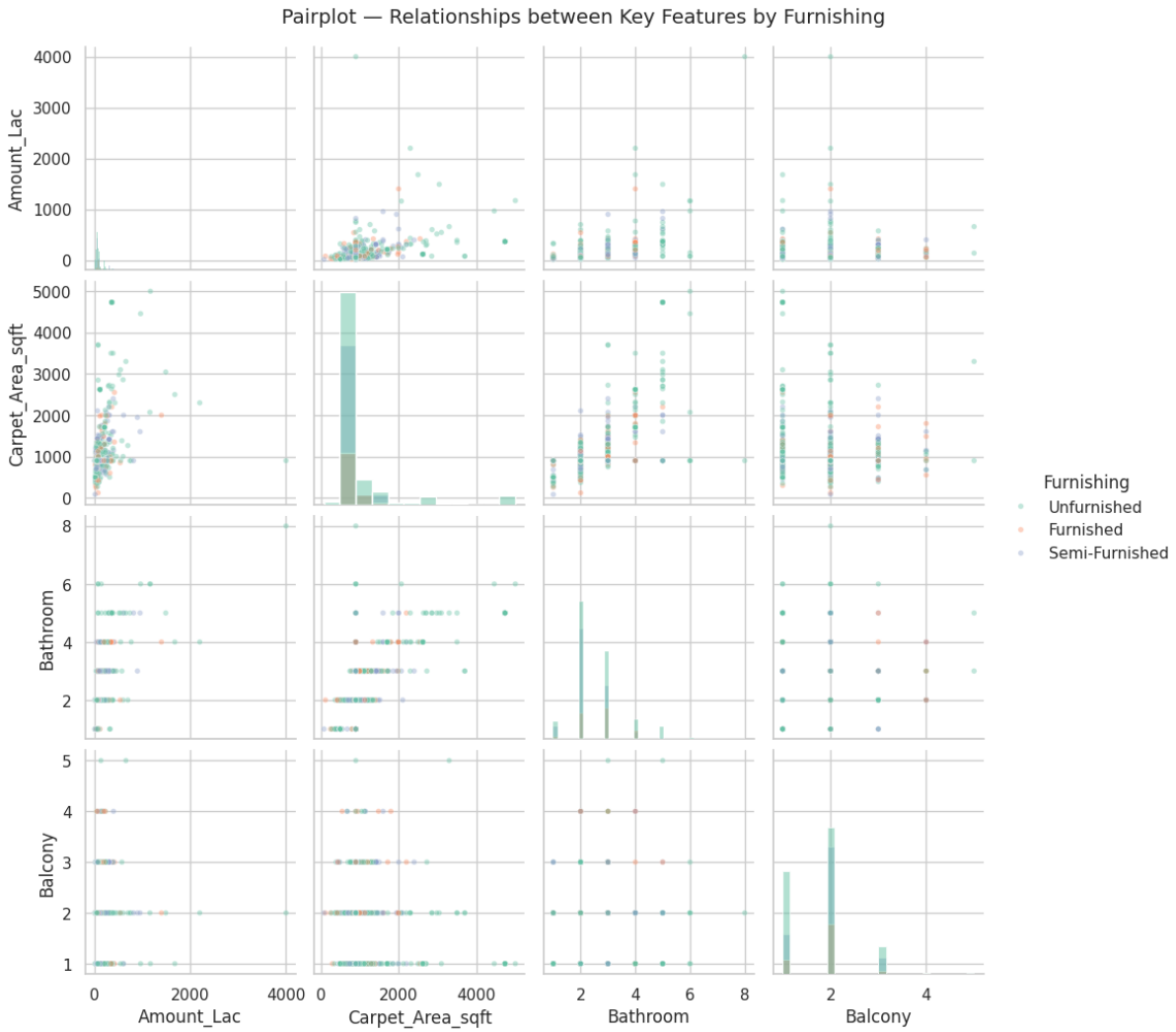


Figure 4.5 – Pairplot of Key Features coloured by Furnishing Status

Interpretation: The pairplot reveals how furnishing status cuts across all pairwise feature relationships simultaneously. Fully furnished properties (shown in one colour cluster) tend to occupy the upper ranges of both price and carpet area, suggesting they are concentrated in the premium segment. Semi-furnished and unfurnished properties overlap considerably in the mid-range, indicating that furnishing alone does not strongly separate these two groups in terms of price. The diagonal histograms confirm that Amount_Lac and Carpet_Area_sqft are right-skewed for all furnishing categories, while Bathroom and Balcony counts form discrete distributions. Overall, the pairplot confirms that Carpet_Area_sqft and Amount_Lac share the strongest pairwise correlation across all furnishing groups.

Grouped Bar Chart — Average Price by Furnishing x Transaction

```
grouped =
df_clean.groupby(['Furnishing','Transaction'])['Amount_Lac'].mean().unstack()
grouped.plot(kind='bar', ax=ax, colormap='viridis')
```

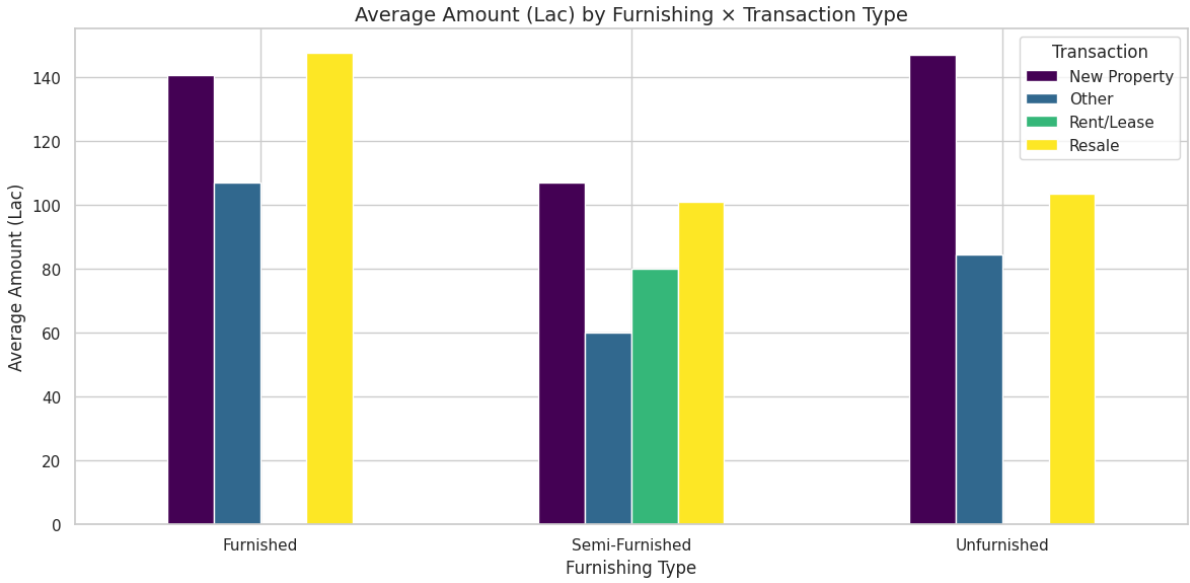


Figure 4.6 – Average Property Price by Furnishing Type and Transaction Type

Interpretation: The grouped bar chart provides a two-dimensional view of how both furnishing status and transaction type simultaneously affect the average property price. New properties command a notably higher average price than resale listings across all furnishing categories, reflecting the premium buyers pay for modern construction, builder warranties, and the absence of prior occupancy. Within each transaction type, fully furnished properties consistently achieve the highest average prices, as the cost of fittings and appliances is bundled into the listing price. Unfurnished new properties occupy a middle ground — priced higher than resale equivalents but lower than furnished counterparts. Semi-furnished resale properties represent the most affordable combination and account for the bulk of the transaction volume in the dataset. These cross-category patterns underscore that pricing strategy should consider both furnishing and transaction type jointly rather than either in isolation.

Chapter 5: Data Transformation and Feature Engineering

Before advanced analysis or modelling, the data needs to be in a form that algorithms can work with. This means converting categorical labels to numbers, scaling numeric features to a common range, correcting skewed distributions, and creating new derived features that capture additional signal.

5.1 Encoding Categorical Variables

Label Encoding is used for columns with a natural order or limited categories (Status, Transaction, Furnishing, Ownership). One-Hot Encoding is applied to the 'facing' column, which has no inherent order. High-cardinality text columns like location and Society are dropped at this stage.

```
from sklearn.preprocessing import LabelEncoder

le_cols = ['Status', 'Transaction', 'Furnishing', 'Ownership']
for col in le_cols:
    le = LabelEncoder()
    df_encoded[col + '_LE'] = le.fit_transform(df_encoded[col].astype(str))

# One-Hot Encoding for 'facing'
df_encoded = pd.get_dummies(df_encoded, columns=['facing'], drop_first=True)

# Drop high-cardinality columns
df_encoded.drop(columns=['location','overlooking','Society'] + le_cols,
inplace=True)
```

5.2 Feature Scaling

StandardScaler (zero mean, unit variance) is applied to all numeric columns. This ensures no single feature dominates due to scale differences — particularly important for PCA and distance-based methods.

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
df_scaled[numeric_for_scale] = scaler.fit_transform(df_encoded[numeric_for_scale])
```

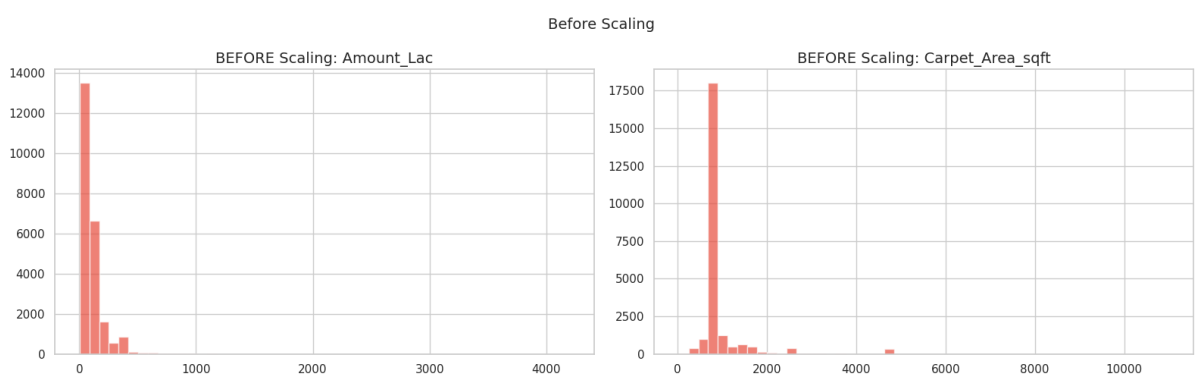


Figure 5.1 – Before StandardScaler: Original distributions of Amount_Lac and Carpet_Area_sqft

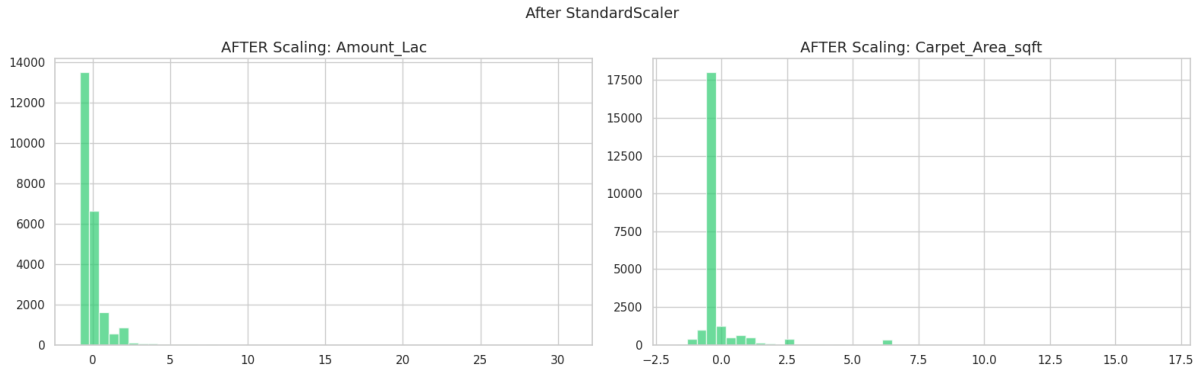


Figure 5.2 – After StandardScaler: Scaled distributions (zero mean, unit variance)

Interpretation: Figures 5.1 and 5.2 together illustrate the effect of StandardScaler on the two most important numeric features. Before scaling (Figure 5.1), Amount_Lac spans a wide range of roughly 5 to 5,000 Lac, while Carpet_Area_sqft ranges from approximately 100 to 10,000 sqft — two entirely incomparable scales. After scaling (Figure 5.2), both features are centred at zero with unit variance, placing them on a common footing. This transformation is critical for any algorithm that uses distance or variance — such as PCA or k-means clustering — since without it, Amount_Lac would dominate purely due to its larger numeric range. The shape of each distribution is preserved; only the axis scale changes.

5.3 Handling Skewness — Log Transform

Price and area columns are highly right-skewed. A log1p transform (log(1 + x)) compresses the long tail and produces distributions closer to normal, which benefits both visualisation and any statistical methods that assume normality.

```
skew_cols = ['Amount_Lac', 'Price (in rupees)', 'Carpet_Area_sqft']
for col in skew_cols:
    log_data = np.log1p(df_encoded[col].clip(lower=0))
    # Before skew vs. After skew printed for comparison
```

Figure 5.3 – Skewness Correction: Before (red) and After (green) Log Transform for Amount_Lac, Price per sqft, and Carpet Area

Interpretation: The before-and-after comparison in Figure 5.3 demonstrates the effectiveness of the log1p transformation in correcting right-skewness. The red distributions (pre-transform) exhibit pronounced long tails to the right — a small number of extremely expensive or large properties stretch the distribution far beyond the bulk of the data. After applying log1p (green distributions), all three features — Amount_Lac, Price per sqft, and Carpet Area — adopt a shape that is much closer to a symmetric, bell-shaped distribution. This correction is important for two reasons: it prevents high-value outliers from exerting disproportionate influence in statistical summaries, and it makes the data more suitable for methods that assume normality, such as linear regression or Gaussian-based outlier detection. The skewness values printed in the code output confirm a substantial reduction in skewness for all three columns post-transform.

5.4 Feature Engineering — New Derived Features

```
# Price normalised by number of bathrooms
df_feat['Price_per_Bathroom'] = df_feat['Amount_Lac'] /
df_feat['Bathroom'].replace(0, 1)

# Relative floor position (0 = ground, 1 = top floor)
df_feat['Floor_Ratio'] = df_feat['Current_Floor'] /
df_feat['Total_Floors'].replace(0, 1)

# Composite amenity score
df_feat['Amenity_Score'] = df_feat['Bathroom'] + df_feat['Balcony'] +
df_feat['Car_Parking_Count']
```

Feature	Formula	Purpose
Price_per_Bathroom	Amount_Lac / Bathroom	Space-adjusted value proxy
Floor_Ratio	Current_Floor / Total_Floors	Relative floor position in building
Amenity_Score	Bathroom + Balcony + Car_Parking_Count	Composite amenity quality metric

Chapter 6: Advanced Analysis

6.1 Correlation Analysis

A correlation heatmap shows pairwise relationships between all numeric features. This helps identify which features are most closely linked to price and which are redundant.

```
numeric_df = df_feat.select_dtypes(include=[np.number])
corr_matrix = numeric_df.corr()

mask = np.triu(np.ones_like(corr_matrix, dtype=bool))
sns.heatmap(corr_matrix, mask=mask, annot=True, fmt='.2f',
            cmap='RdBu_r', center=0, square=True, linewidths=0.5)
```

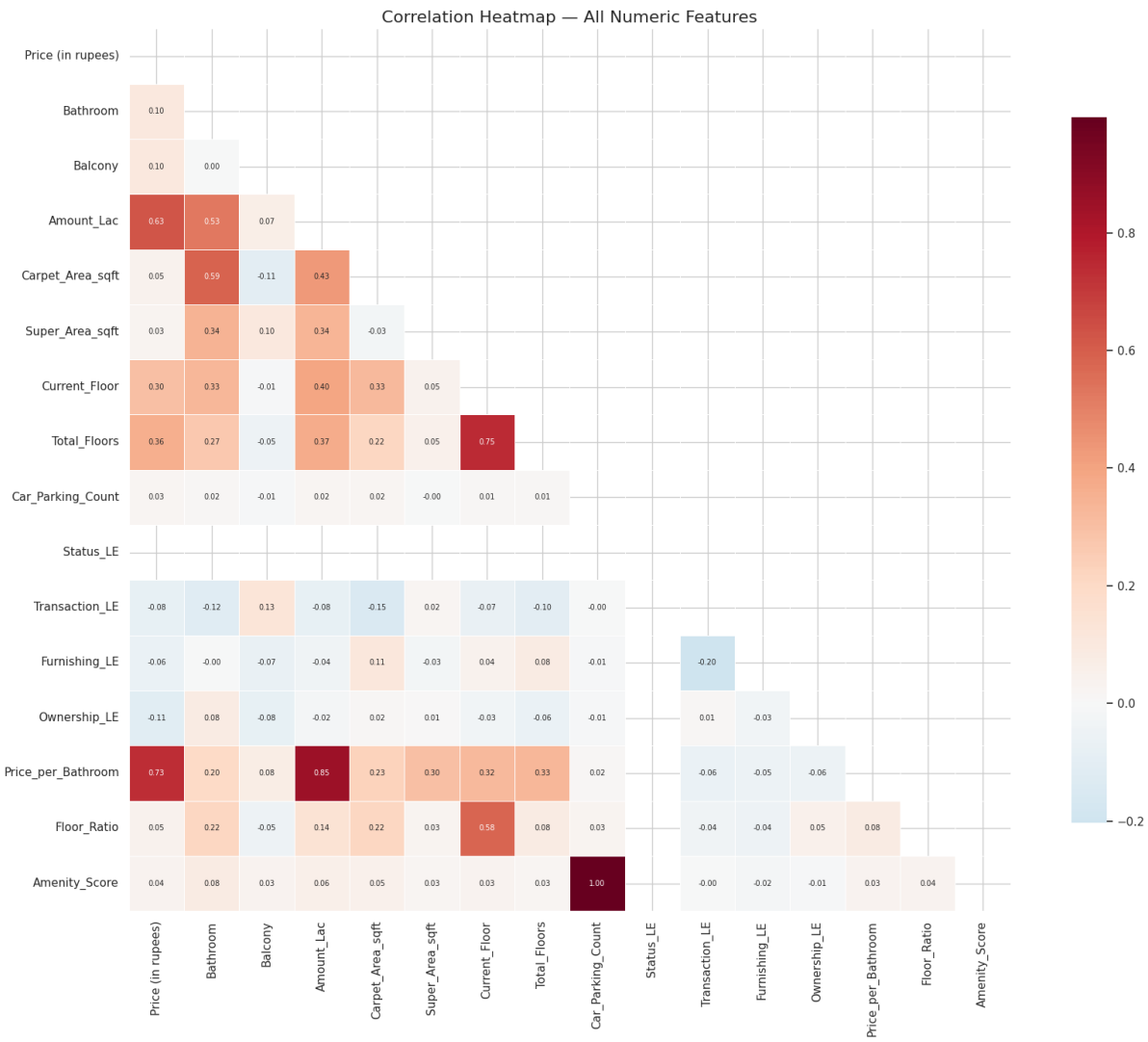


Figure 6.1 – Correlation Heatmap: All Numeric Features (lower triangle, RdBu_r palette)

Key correlations identified:

- Amount_Lac ↔ Carpet_Area_sqft: Strong positive — larger flats cost more.

- Amount_Lac ↔ Bathroom: Moderate positive — bathroom count reflects overall size.
- Amenity_Score ↔ Amount_Lac: Meaningful positive — more amenities = higher price.

6.2 Feature Selection

```
target_corr =
corr_matrix['Amount_Lac'].drop('Amount_Lac').abs().sort_values(ascending=False)
ax.barh(target_corr.index[:15], target_corr.values[:15])
ax.axvline(x=0.3, color='green', linestyle='--', label='Strong (>0.3)')
ax.axvline(x=0.1, color='orange', linestyle='--', label='Moderate (>0.1)')
```

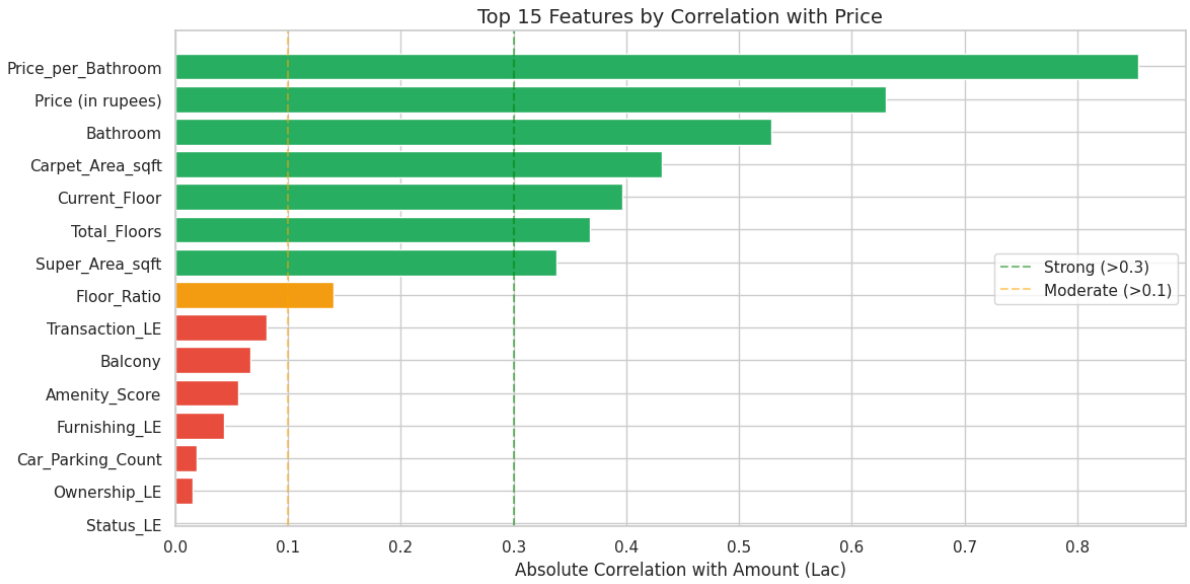


Figure 6.2 – Top 15 Features by Absolute Correlation with Property Price (Amount_Lac)

Carpet Area, Bathroom count, and the engineered Amenity Score consistently appear as the top predictors of property price. These features should be prioritised in any modelling exercise.

6.3 Principal Component Analysis (PCA)

PCA reduces the dataset to its most informative dimensions. It is useful for understanding the effective complexity of the data and for removing redundant features before modelling.

Explained Variance

```
from sklearn.decomposition import PCA
pca_full = PCA()
pca_full.fit(pca_data)
cumulative = np.cumsum(pca_full.explained_variance_ratio_)
n_90 = np.argmax(cumulative >= 0.90) + 1
n_95 = np.argmax(cumulative >= 0.95) + 1
print(f'Components for 90% variance: {n_90}')
print(f'Components for 95% variance: {n_95}')
```

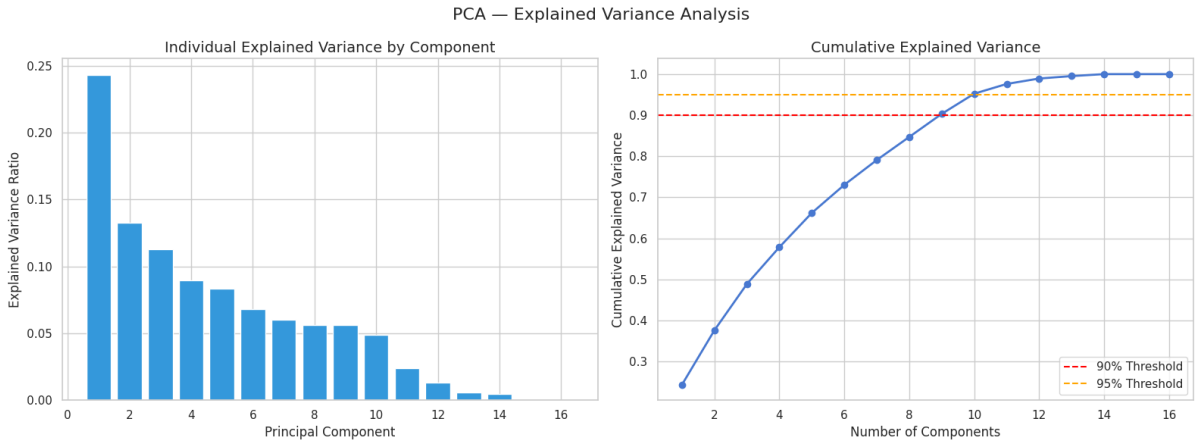


Figure 6.3 – PCA Explained Variance: Individual (left) and Cumulative (right) with 90% and 95% thresholds

Interpretation: The individual explained variance chart (left) shows a steep drop-off after the first few principal components, with the first component alone capturing the largest share of variance. This pattern, known as an elbow curve, indicates that the data has a small number of dominant dimensions that summarise most of the information. The cumulative chart (right) reveals that approximately 5–7 components are sufficient to explain 90% of total variance, and roughly 8–10 components capture 95%. Given that the dataset originally has 16+ numeric features, this is a significant reduction. It confirms that many of the original features are highly correlated or redundant — particularly those related to property size (Carpet Area, Super Area, Bathroom count), which all tend to increase together. In a future modelling exercise, retaining only the top 7–8 principal components would substantially reduce computational cost while preserving nearly all predictive information.

2D PCA Scatter Plot

```
pca_2d = PCA(n_components=2)
pca_result = pca_2d.fit_transform(pca_data)
quartiles = pd.qcut(amount_col, q=4, labels=['Budget','Mid-
Range','Premium','Luxury'])
# Colour-coded scatter by price quartile
```

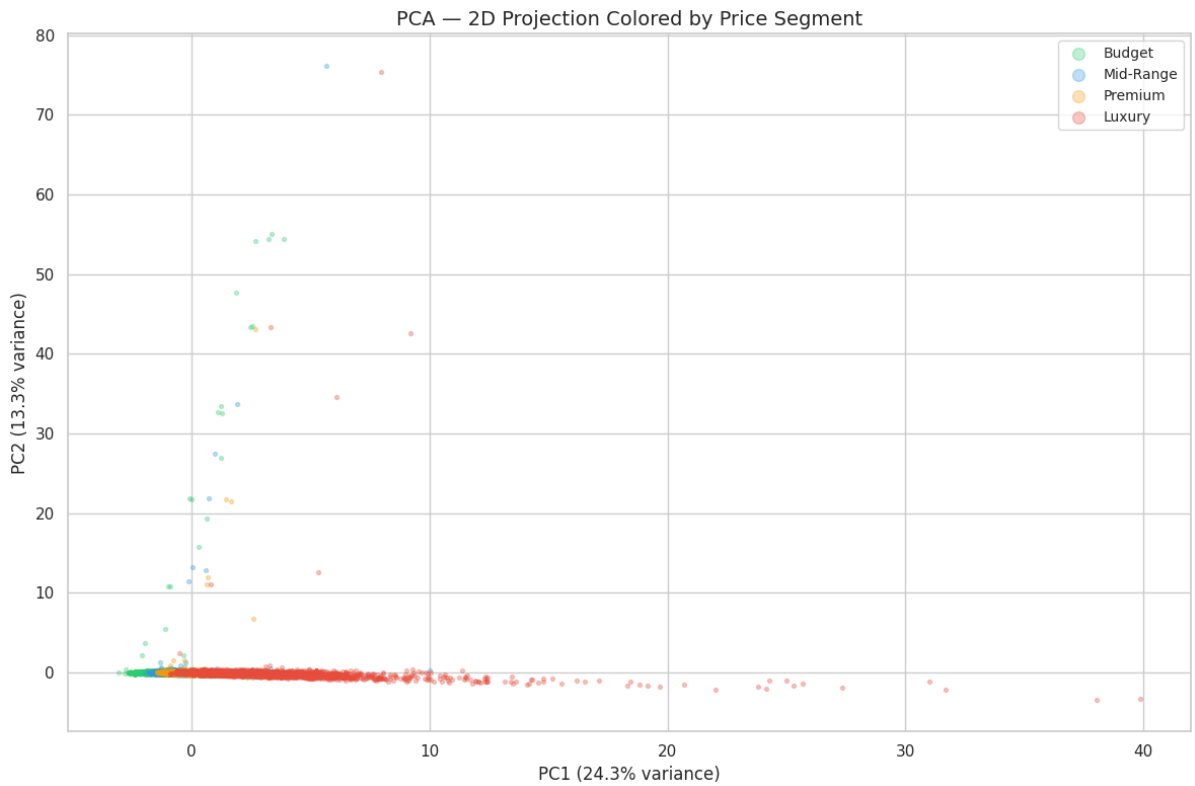


Figure 6.4 – PCA 2D Projection coloured by Price Segment (Budget / Mid-Range / Premium / Luxury)

Interpretation:

- A small number of components capture most of the variance, confirming the data is not as complex as its 21 features suggest.
- PC1 separates Budget and Luxury segments clearly — it primarily represents overall property size and value.
- Mid-range properties overlap, showing that mid-tier listings are harder to differentiate on features alone.

Chapter 7: Visualization and Dashboard

7.1 Tool Used

Power BI was used to build the interactive dashboard. It was selected for its ease of integration with large datasets, its drag-and-drop visualisation builder, and its ability to produce publish-ready dashboards without writing additional code.

7.2 Types of Visualisations Used

#	Chart Type	What It Shows
1	Bar Chart	Average property price by city/location
2	Scatter Plot	Carpet Area vs Total Price with furnishing colour coding
3	Pie / Donut Chart	Share of listings by Furnishing Status
4	Box Plot / Violin	Price distribution by BHK category
5	Heatmap (Matrix)	Correlation between numeric features
6	Line Chart	Price trend across floor levels

7.3 Dashboard Explanation

The Power BI dashboard is structured around a single-page layout with interactive filters. Users can filter by city, furnishing type, and price range using slicers on the left panel. Each chart updates dynamically based on the selection. The main KPI cards at the top show total listings, average price, and average carpet area at a glance.

7.4 Dashboard Screenshot

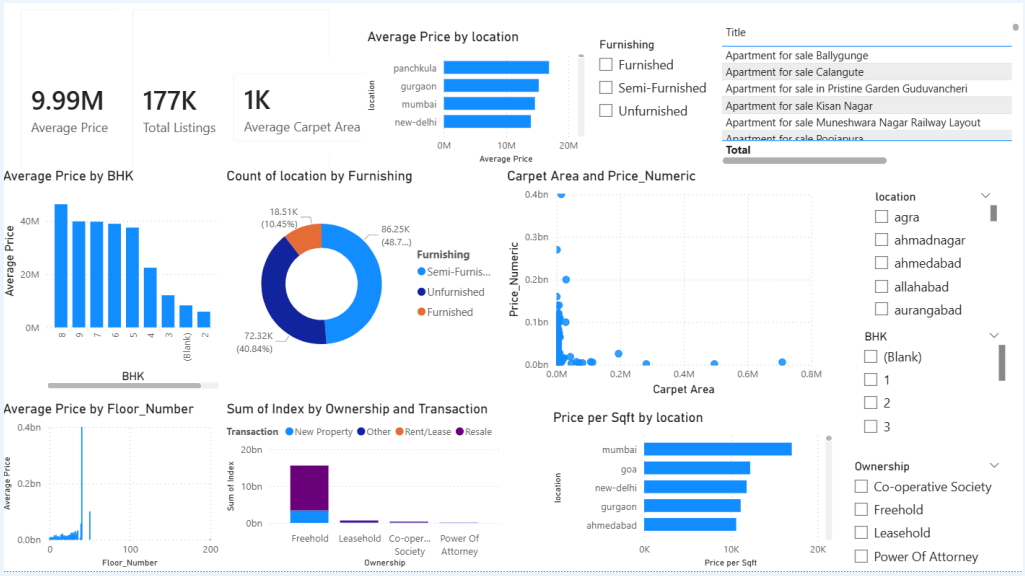


Figure 7.1 – Interactive Power BI Dashboard with KPI cards, slicers, and 6 chart types

Chapter 8: Insights and Storytelling

8.1 Key Findings from Analysis

The analysis of the housing dataset revealed several important patterns:

- The average property price is approximately ₹9.99 million, reflecting a mid-to-high value real estate market.
- The dataset contains around 177,000 usable listings after cleaning — sufficient for robust analysis.
- Average carpet area is ~1,000 sqft, indicating most listings are mid-sized 2 BHK units.
- Location matters significantly — cities like Panchkula, Gurgaon, Mumbai, and New Delhi show higher average prices per sqft.
- Semi-furnished properties make up the largest share (~48%), followed by unfurnished (~40%) and fully furnished (~10%).
- A positive correlation between carpet area and price is consistent across the dataset.

8.2 Patterns and Trends Identified

BHK vs Price

- Properties with 6–8 BHK have significantly higher prices — these are premium luxury listings.
- 1–3 BHK units dominate the volume and are more affordable, representing the mass market.

Location-Based Pricing

- Metro cities like Mumbai and Delhi show higher price per sqft due to high demand and limited space.
- Cities like Ahmedabad and Gurgaon offer relatively moderate pricing while still being urban markets.

Carpet Area vs Price Relationship

- A general increasing trend holds: larger flats tend to cost more.
- Some large-area properties have surprisingly low prices — indicating location or accessibility factors at play.

Floor Number Impact

- Properties on certain floors (typically mid-to-high in apartment complexes) show price premiums, reflecting buyer preference for views and privacy.

8.3 Business and Social Insights

- Location is the single most influential factor in determining property prices — even more than size.
- High price per sqft in metro cities reflects strong urban demand and restricted land supply.
- The dominance of semi-furnished homes shows buyer preference for move-in-ready properties with room for personalisation.

- The BHK-price relationship enables clear market segmentation: lower BHK targets the mass market, higher BHK targets premium buyers.
- Outliers in the price vs area relationship highlight that factors like connectivity, amenities, and locality significantly influence pricing beyond physical size alone.

8.4 Decision-Making Based on Results

Stakeholder	Recommendation
Buyers	Compare price per sqft rather than total price for fair cross-city comparison. Consider emerging cities (Thane, Navi Mumbai) for better affordability.
Sellers & Builders	Invest in semi-furnished 2–3 BHK units — highest demand segment. Target metro-adjacent markets for premium projects.
Real Estate Companies	Use location-based pricing strategies and data-driven valuation. Focus marketing on mid-range BHK properties with the highest transaction volume.
Data & Modelling Teams	Use Carpet Area, Bathroom count, and Amenity Score as primary features. Apply PCA to reduce dimensionality while retaining 90%+ variance.

Chapter 9: Conclusion

9.1 Summary of the Project

This project carried out a complete data science workflow on a real-world Indian housing dataset. Starting from raw, messy CSV data, we cleaned and transformed the dataset, explored it visually, engineered new features, and conducted advanced statistical analysis. The work culminated in an interactive Power BI dashboard and a set of actionable insights for buyers, sellers, and analysts.

9.2 Outcomes Achieved

- Successfully cleaned a 187,531-row dataset with mixed formats, missing values, and irregular structure.
- Identified Carpet Area, Bathroom count, and Location as the primary price drivers through EDA and correlation analysis.
- Demonstrated through PCA that the effective dimensionality of the data is significantly lower than its 21 features.
- Built an interactive Power BI dashboard presenting 6+ chart types for non-technical stakeholders.
- Derived clear business recommendations covering buyers, sellers, and real estate companies.

9.3 Limitations

- The dataset is limited to listings in and around Thane — findings may not generalise directly to other Indian cities.
- High-cardinality columns like 'location' and 'Society' were dropped due to complexity, losing potentially valuable geographic signal.
- The dataset reflects listing prices, not actual transaction prices — these can differ significantly.
- No time dimension is available, so trends over time cannot be studied.

9.4 Future Scope

- Integrate time-series data to model price trends across quarters or years.
- Build a machine learning regression model (Random Forest, XGBoost) to predict property prices.
- Apply NLP to the Description column to extract features like 'near metro' or 'sea view'.
- Extend the analysis to multiple cities for a pan-India comparison dashboard.
- Deploy an interactive price estimator as a web application using Streamlit or Flask.

Chapter 10: References

Dataset Sources

Kaggle – House Price Dataset: kaggle.com/datasets/juhibhojani/house-price

Tools and Libraries Used

Tool / Library	Purpose
Python 3.x	Primary programming language
Pandas	Data loading, cleaning, manipulation
NumPy	Numerical operations and array processing
Matplotlib	Base plotting library
Seaborn	Statistical visualisations
Scikit-learn	Encoding, scaling, PCA
SciPy	Z-score outlier detection
Google Colab	Cloud-based notebook environment
Power BI	Interactive dashboard creation
GitHub	Version control and collaboration

Websites and References

- Pandas CSV parsing documentation: https://pandas.pydata.org/docs/reference/api/pandas.read_csv.html
- Stack Overflow – CSV parsing errors: https://stackoverflow.com/questions/52105659
- Scikit-learn – PCA documentation: https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.PCA.html
- Seaborn documentation: https://seaborn.pydata.org/

Appendix

GitHub Repository

https://github.com/jeevaprakash-S/24ADI204_DSV_TEAM-3

Team Contributions

Name	Roll No.	Contribution
Hariharan V S	24BAD030	Colab notebook, preprocessing, dashboard
Jeevaprakash S	24BAD048	GitHub setup, report writing, EDA
Jessica Sam B	24BAD049	Report writing, visualisations, insights/storytelling
Kavin T	24BAD058	Dataset analysis, code, feature engineering